

Quickly make a pairs trading strategy with 500 stocks



September 14, 2024



The PyQuant Newsletter

PQN

#113

Quickly make a pairs trading strategy with 500 stocks

Real Python code. No jargon. 5 minutes to read. pyquantnews.com | x.com/pyquantnews

Quickly make a pairs trading strategy with 500 stocks

Pairs trading is a market-neutral strategy used to exploit the relative value discrepancies between two correlated stocks.

We buy an “undervalued” stock and sell an “overvalued” one with the assumption the relationship will mean revert.

Free Algorithmic Trading Course

Get a free, 7-day email course to get started with to build, backtest, and automate trading strategies with Python

"I can't believe you just give this away for free." - Tim, Independent Trader

Send me Lesson #1



Pairs trading involves some basic statistics which are easy to implement in Python.

In today's newsletter, we'll build a basic pairs trading strategy and backtest it with VectorBT Pro.

Let's go!

Quickly make a pairs trading strategy with 500 stocks

In practice, pairs trading starts with identifying two stocks with a strong historical correlation. Traders monitor the price spread between these stocks and trade them when deviations occur.

Tools like cointegration tests help in this process.

Cointegration looks to see how stable the relationship between the pairs is. Statistical arbitrage, like pairs trading, uses models to pinpoint mispricings, incorporating factors like momentum and volatility.

Spread is the relative performance between the assets. Whenever the spread increases and reaches a threshold, we take a long position in the underperformer and a short position in the overachiever.

Such a threshold is usually set to a number of standard deviations from the mean.

Let's see how it works with Python.

Imports and set up

You'll need [VectorBT Pro](#) for today's newsletter. After you install it, import pandas, SciPy, Statsmodels, and NumPy.

Great example of the Python Quant Stack in action!

```
1 from vectorbtpro import *
2 import pandas as pd
3 import scipy.stats as st
4 import statsmodels.tsa.stattools as ts
5 import numpy as np
6 import warnings
7 warnings.filterwarnings("ignore")
```

Next, let's get the tickers for the S&P500 index and store the price data and set up some variables

```
1 sp500_tickers =
pd.read_html('https://en.wikipedia.org/wiki/List_of_S%26P_500_companies')
[0]['Symbol'].tolist()
2
3 COINT_FILE = "coint_pvalues.pickle"
4 POOL_FILE = "data_pool.h5"
5 START = "2015-01-01"
6 END = "2023-12-31"
```

Load historic price data

We'll use VectorBT to grab price data for free.

```
1 if not vbt.file_exists(POOL_FILE):
2     with vbt.ProgressBar(total=len(sp500_tickers)) as pbar:
3         collected = 0
4         for symbol in sp500_tickers:
5             try:
6                 data = vbt.YFData.pull(
7                     symbol,
8                     start=START,
9                     end=END,
10                    silence_warnings=True,
11                )
12                data.to_hdf(POOL_FILE)
13                collected += 1
14            except:
15                pass
16            pbar.set_prefix(f"{symbol} ({collected})")
17            pbar.update()
```

We load the S&P 500 tickers from Wikipedia using pandas. Then we start collecting historical data for each ticker using the VectorBT's YFData.pull method.

The collected data is then saved into an HDF5 file so we don't have to hit the network every time.

Next, we load the saved data, filter out any symbols with missing data, and keep only valid symbols.

```
1 data = vbt.HDFData.pull(  
2     POOL_FILE,  
3     start=START,  
4     end=END,  
5     silence_warnings=True  
6 )  
7  
8 data = data.select_symbols([  
9     k  
10    for k, v in data.data.items()  
11    if not v.isnull().any().any()  
12 ])
```

We load the saved data from the HDF5 file. We then iterate over each symbol's data and check for missing values. If a symbol has any missing values, it is excluded.

Identify cointegrated pairs of stocks

Now, we identify pairs of stocks that are cointegrated.

Cointegration helps us find pairs of stocks that have a stable relationship over time, which is essential for pairs trading strategies.

Warning: This can take a long time!

```
1 @vbt.parameterized(  
2     merge_func="concat",  
3     engine="pathos",  
4     distribute="chunks",
```

```

5     n_chunks="auto"
6 )
7 def coint_pvalue(close, s1, s2):
8     return ts.coint(np.log(close[s1]), np.log(close[s2]))[1]
9
10 if not vbt.file_exists(COINT_FILE):
11     coint_pvalues = coint_pvalue(
12         data.close,
13         vbt.Param(data.symbols, condition="s1 != s2"),
14         vbt.Param(data.symbols)
15     )
16     vbt.save(coint_pvalues, COINT_FILE)
17 else:
18     coint_pvalues = vbt.load(COINT_FILE)
19
20 coint_pvalues = coint_pvalues.sort_values()
21 coint_pvalues.head(20)

```

We define a function to calculate the cointegration p-value between two price series. We use the parameterized decorator to parallelize the computation.

If the cointegration p-values file does not exist, we calculate the p-values for all pairs of stocks and save the results. Otherwise, we load the saved p-values.

We then sort the p-values in ascending order.

Create the trading strategy

Finally, we choose a specific pair of stocks, analyze their price relationship, and build the strategy. We'll use the two stocks with the highest statistical significance.

```

1 S1, S2 = "WYNN", "DVN"
2
3 data = vbt.YFData.pull(
4     [S1, S2],
5     start=START,
6     end=END,
7     silence_warnings=True,
8 )

```

Once we select the stocks, we pull historic price data for just those two.

Next, run a regression to get the zscore.

```
1 UPPER = st.norm.ppf(1 - 0.05 / 2)
2 LOWER = -st.norm.ppf(1 - 0.05 / 2)
3
4 S1_close = data.get("Close", S1)
5 S2_close = data.get("Close", S2)
6 ols = vbt.OLS.run(S1_close, S2_close, window=vbt.Default(21))
7 spread = ols.error.rename("Spread")
8 zscore = ols.zscore.rename("Z-score")
```

We use the prediction error of the ordinary least squares (OLS), that is, the difference between the true and predicted value.

Finally, we can run the backtest

```
1 upper_crossed = zscore.vbt.crossed_above(UPPER)
2 lower_crossed = zscore.vbt.crossed_below(LOWER)
3
4 long_entries = data.symbol_wrapper.fill(False)
5 short_entries = data.symbol_wrapper.fill(False)
6
7 short_entries.loc[upper_crossed, S1] = True
8 long_entries.loc[upper_crossed, S2] = True
9 long_entries.loc[lower_crossed, S1] = True
10 short_entries.loc[lower_crossed, S2] = True
11
12 pf = vbt.Portfolio.from_signals(
13     data,
14     entries=long_entries,
15     short_entries=short_entries,
16     size=10,
17     size_type="valuepercent100",
18     group_by=True,
19     cash_sharing=True,
20     call_seq="auto"
21 )
22
23 pf.stats()
```

You'll see that this pair seems to outperform the benchmark over the analysis period.

Your next steps

It's important to conduct a walk forward analysis for a strategy like this. That way you can get a feel for potential overfitting.

(I just ran out of room in this newsletter!)

You can also try changing the selected stock pair to explore different cointegrated pairs. Experiment with different window sizes in the OLS regression to find an optimal setting for your strategy.